

DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

Anonymous authors
Anonymous submission

ABSTRACT

Parametric imitation learning with behavior cloning is vulnerable to poor closed-loop generalization when compounding prediction errors induce visits to states outside the empirical demonstration distribution. We show that retaining the training corpus as an inference-time memory and querying it with a semi-parametric retrieval procedure can mitigate this failure mode. We present **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, a semi-parametric retrieval-based imitation learning method that reparameterizes policy prediction around local neighborhood structure rather than direct global state-to-action regression. For each query state, DARP retrieves k expert nearest neighbors and conditions the action predictor on the neighbor states, their associated expert actions, and relative difference vectors between neighbor states and the query state. The method preserves the information assumptions of standard behavior cloning: it requires no additional data collection, no online expert feedback, and no task-specific side information. Across continuous-control and robotic-manipulation domains, including settings with high-dimensional visual representations, DARP yields consistent 15-46% performance improvements over standard behavior cloning. Code and demos will be released upon acceptance.

1 INTRODUCTION

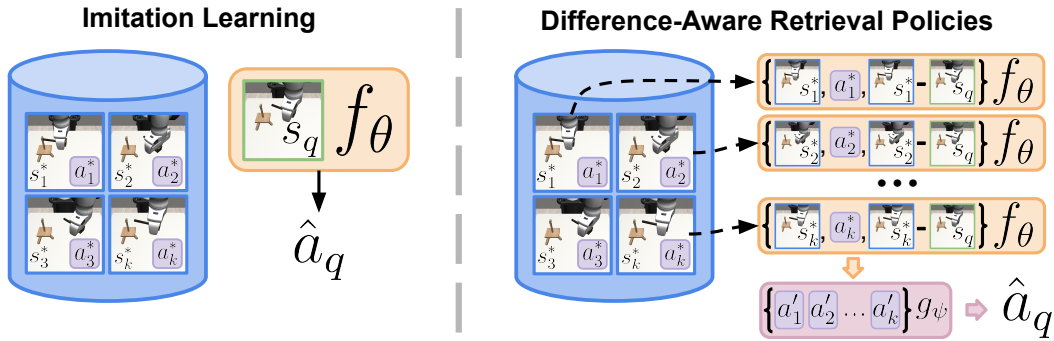


Figure 1: **Overview of DARP:** Standard BC (left) fits a parametric mapping from expert states to actions and discards the expert dataset after training. DARP (right) instead retains the expert state-action corpus at inference time, retrieves a local neighborhood around each query state s_q , and evaluates a difference-aware predictor parameterized by relative vectors between the query and retrieved neighbors.

Imitation learning through behavior cloning (BC) (Pomerleau, 1991) provides a simple supervised-learning interface for acquiring complex robotic behaviors from expert demonstrations (Zhao et al., 2023; Chi et al., 2024; Black et al., 2024; Chung et al., 2014). However, the closed-loop execution of a BC policy remains fragile, particularly in long-horizon tasks (Ross et al., 2011). The central mechanism is *covariate shift*: small action errors perturb the state trajectory, subsequent decisions are evaluated on states that are underrepresented in the demonstration set, and the resulting prediction errors compound over time (Spencer et al., 2021; Ross et al., 2011). In these out-of-distribution

regions, an unconstrained BC policy can exhibit high-variance and unstable action outputs, frequently causing task failure.

A substantial body of prior work attempts to reduce compounding error (Ross et al., 2011; Venkatraman et al., 2015; Ke et al., 2024b; Levine et al., 2020). Many such methods, however, enlarge the standard BC information model by requiring simulator access, interactive expert queries, large quantities of suboptimal data, or task-specific structural assumptions. We instead retain the pure BC setting: the learner receives only expert state–action pairs and does not obtain additional supervision during training or deployment. Under this restriction, the relevant technical question is: *can the original demonstration dataset be used more effectively to reduce the variance of BC policies?*

From a statistical perspective, BC optimizes supervised risk only on expert states. This objective constrains prediction bias on the empirical training distribution, but it does not regularize policy variation in low-density regions that may be reached during closed-loop rollouts. Consequently, the learned function may oscillate sharply away from sampled expert states. A principled remedy is to impose *smoothness*, requiring nearby states to induce similar predicted actions (Kobayashi, 2022; Asadi et al., 2018; Ke et al., 2024a; Chen et al., 2024). Such a constraint suppresses spurious local fluctuations and improves rollout stability. Existing mechanisms for inducing smoothness are discussed in Section 4.

These approaches can be effective, but each introduces a distinct limitation: data augmentation does not guarantee local consistency, global continuity priors may merge distinct behaviors, temporal penalties constrain only trajectory-time variation rather than state-space variation, and explicit graph regularizers require additional smoothness hyperparameters.

A complementary methodological distinction separates global and local predictors. “Global” supervised models (Black et al., 2024; Zhao et al., 2023; Chi et al., 2024) compress the demonstration corpus into a single parametric function, which can become brittle under distribution shift. “Local” methods (Pari et al., 2022; Mansimov & Cho, 2018; Salzberg & Aha, 1994) instead condition predictions on the empirical structure of the dataset by retrieving similar states and computing outputs through non-parametric or semi-parametric operations over nearby expert behavior. Locality improves robustness by anchoring predictions to observed examples, but it also has technical limitations: performance depends on the retrieval metric, naive neighborhood averaging can blur distinct action modes, and representing neighbors only by absolute state coordinates can restrict generalization.

We introduce **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, which combines retrieval-based locality with the stability properties of regularized parametric policy learning. At inference time, rather than producing an action solely by applying a feedforward parametric policy to the current query state, DARP (Fig. 1) first retrieves k neighbors from the training corpus. It then predicts k candidate actions conditioned on tuples consisting of the neighbor state, the associated expert action, and the difference vector from the query state. A permutation-invariant aggregation operator maps these candidate actions to a single executable prediction. This architecture grounds predictions in the observed expert corpus through non-parametric retrieval while inducing local consistency through a parametric predictor conditioned on relative neighborhood geometry. We show that this construction reduces variance without changing the assumptions of standard Behavior Cloning: it requires no additional data, online supervision, or task-specific knowledge. Spectrally, the resulting neighbor aggregation approximates a Laplacian filter on the k -NN (k -nearest neighbor) graph of expert states, yielding a parameter-free smoothing operation adapted to the dataset’s local density and geometry.

We provide theoretical and empirical evidence that, under the same information requirements as behavior cloning, DARP improves policy performance by reducing estimator variance and increasing robustness to distribution shift. The analysis formalizes the connection to Laplacian regularization and shows that DARP implicitly applies a fixed low-pass spectral filter that attenuates high-frequency variance. Empirically, DARP achieves 15–46% gains over standard behavior cloning across continuous control (MuJoCo), robotic manipulation (Robosuite, Robocasa), and high-dimensional visual imitation tasks (Robosuite with image state). The method is a general semi-parametric architecture that extends naturally to image-based domains and richer policy classes, including transformers and Gaussian mixture models. We further report architectural ablations that isolate the contribution of the proposed representation and aggregation design, providing general guidance for retrieval-based algorithms in sequential decision-making.

2 DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

This section defines a class of imitation-learning algorithms that combine the representational capacity of “global” parametric policies with the data-adaptive behavior of “local” retrieval methods. We introduce an architecture and training objective that operate under the same data assumptions as standard behavior cloning while yielding theoretical and empirical improvements. We first derive the relevant regularization guarantees from first principles. Readers primarily interested in the practical algorithm may proceed directly to Section 2.5, which gives a self-contained description of DARP before the empirical evaluation in Section 3.

We begin by specifying the imitation-learning problem setting (Section 2.1) and then analyze an explicitly regularized variant of imitation learning (Section 2.2) that uses neighborhood structure in the expert dataset to improve variance, generalization, and stability. Section 2.3 shows that the same smoothing effect can be induced through the policy *architecture*, rather than by adding a regularization term to the loss. Section 2.4 then instantiates these principles as DARP, a semi-parametric retrieval-augmented policy architecture compatible with modern neural networks and generative policy models.

2.1 PRELIMINARIES: BEHAVIOR CLONING FOR IMITATION LEARNING

We consider the standard imitation-learning setting, formalized as a finite-horizon Markov Decision Process (MDP) $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, P_0\}$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, and P_0 denotes the initial-state distribution. A *policy* maps states to action distributions, $f_\theta : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$, in order to optimize task-relevant performance criteria; when referring to the function class abstractly, we omit the parameter subscript and write f . The learner is given expert demonstrations $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$ consisting of state-action pairs, and the notation s^* and a^* is reserved for states and actions drawn from this expert dataset. Behavior cloning (Pomerleau, 1991) fits f_θ by reducing imitation learning to supervised maximum-likelihood estimation, $\arg \max_\theta \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\log(f_\theta(a^* | s^*))]$. Although f_θ may parameterize a rich generative distribution (Lipman et al., 2023; Chi et al., 2024), we initially use a Gaussian parameterization for analytical clarity¹.

2.2 WARM-UP: NEIGHBOR MANIFOLD REGULARIZED IMITATION LEARNING

Behavior cloning minimizes supervised imitation loss only at expert states sampled from $\mathcal{D}^*$. This pointwise objective does not constrain policy behavior at states that depart from the expert-state manifold. During deployment, accumulated action errors can therefore move the agent into out-of-distribution regions where an overparameterized BC policy may produce arbitrary actions (Ross & Bagnell, 2010).

The missing structural constraint is that BC evaluates the function only at individual training states and does not encode relationships among neighboring states and their associated actions. To incorporate the geometry of the empirical state manifold, we introduce an objective with an explicit regularization term that enforces *local consistency*: nearby expert states should receive similar policy predictions. This yields the neighborhood-regularized loss $\mathcal{L}_{\text{MRIL}}$, which augments the standard imitation-learning objective $\mathcal{L}_{\text{BC}}$ with a smoothness penalty $\mathcal{L}_{\text{S}}$ that forces predictions to respect dataset geometry rather than relying solely on pointwise supervision.

$$\mathcal{L}_{\text{MRIL}}(f) = \underbrace{\mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)]}_{\text{supervised risk}(\mathcal{L}_{\text{BC}})} + \underbrace{\lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right]}_{\text{smoothness regularizer}(\mathcal{L}_{\text{S}})}, \quad (1)$$

where $\ell(f(s^*), a^*)$ is the supervised imitation loss, $\mathcal{N}_k(s^*)$ denotes the k -nearest neighbors of s^* in the expert dataset, and $w_i(s^*)$ denotes normalized kernel weights computed from state differences, $w_i(s^*) \propto K_\Delta\left(\frac{\|s_i^* - s^*\|}{h}\right)$. As summarized below and proved in Appendix A.1.1, this term implements a form of *manifold regularization* or Laplacian smoothing: it penalizes high-frequency variation of the learned policy over the k -NN graph induced by expert states. This regularization improves

¹We show that this can be relaxed in Section 2.4

estimator variance, policy stability, and generalization, following the standard graph-based manifold-regularization interpretation (Zhou et al., 2003; Belkin & Niyogi, 2008).

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.
- (ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.
- (iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^2$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s_t) - f(s_t^*)\| = O(\Delta_t)$ with smaller constant.

Theorem 1 establishes that explicitly regularizing the BC objective with local consistency constraints can reduce variance and improve closed-loop stability relative to vanilla BC. The proof is given in Appendix A.1.1. The principal limitation of this construction is operational: the objective introduces an additional hyperparameter λ and requires explicit optimization of a smoothness penalty. The next section shows that an analogous smoothing operator can instead be embedded directly into the architecture.

2.3 IMPLICIT MANIFOLD REGULARIZATION VIA IN-CONTEXT ARCHITECTURES

Although the MRIL objective amortizes local learning and improves over vanilla BC, it has two operational drawbacks. First, it introduces a hyperparameter λ that must balance supervised fit against smoothness. Second, optimizing a modified regularized objective can alter the optimization landscape relative to the standard BC objective. This motivates the architectural question: *can the benefits of MRIL (Eq. 1) be obtained by changing the policy parameterization rather than changing the loss?*

We answer this question by defining a retrieval-based policy architecture that induces an *implicit* manifold-regularization effect (iMRIL) while retaining the standard imitation objective. Under iMRIL, the Laplacian-smoothing effect associated with MRIL is produced by neighborhood aggregation inside the predictor itself (Fig. 2), eliminating λ as a training hyperparameter. Section 2.4 then specializes this construction into the practical DARP algorithm.

² L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

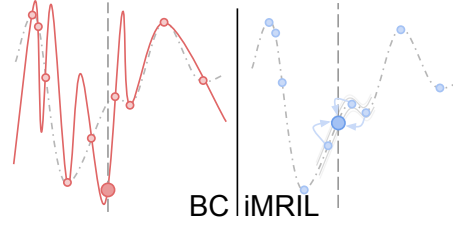


Figure 2: **iMRIL implements Laplacian smoothing through architecture-level neighborhood aggregation.** The induced smoothing reduces variance and enforces local consistency, whereas standard BC does not constrain high-frequency or jagged function approximations between expert states.

iMRIL architecture: The central construction moves the neighborhood-aggregation operation from the objective in Eq. 1 into the action predictor. Instead of learning a feedforward policy $f(s)$ under an explicit smoothness-regularized loss, iMRIL parameterizes the predictor $\hat{f}$ itself as an average over predictions at nearest-neighbor expert states, while preserving the ordinary imitation-learning objective. Specifically, iMRIL learns a per-state predictor f_θ such that $\hat{f}(s^*) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*)$, where the aggregation ranges over training-set neighbor states $\{s_i^*\}_{i \in \mathcal{N}_k(s^*)}$, accurately predicts the expert action a^* . Training optimizes the objective in Eq. 2:

$$\arg \min_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} \left[\left\| \underbrace{\left(\frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*) \right)}_{\hat{f}(s^*)} - a^* \right\|_2 \right] \quad (2)$$

At deployment, a query state s_q is evaluated by retrieving its k nearest neighbors from the training set and computing $\hat{a} = \hat{f}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_\theta(s_i^*)$. Section 2.4 shows that the parameterization of f is empirically consequential and motivates the difference-aware representation used in DARP.

This construction parameterizes the executed predictor $\hat{f}$ as an aggregation of learned neighbor-state predictions $f(s_i^*)$. Because supervision is applied after aggregation, individual neighbor predictions cannot induce arbitrarily nonsmooth executed actions without being averaged through the neighborhood operator. Thus the architecture inherits the local-consistency benefits described in Section 2.2. We formalize this correspondence through a spectral equivalence to Laplacian regularization.

Equivalence between iMRIL and MRIL: Appendix Section A.1 contains the full proof; here we state the main result and give the proof sketch.

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof sketch. We defer full details to Appendix Section A.1.2. The explicit regularizer admits a spectral solution by diagonalizing the k -NN Laplacian, yielding a filter of the form $(1 + \lambda \mu)^{-1}$ for each eigenmode. The implicit objective can be written as neighbor aggregation $\hat{f} = S f$ with $S = D^{-1} A$, the random-walk matrix, which shares the same eigenvectors and applies the fixed filter $1 - \mu$. Both operators are therefore low-pass filters on the graph: low-eigenvalue modes corresponding to smooth variation over the data manifold are largely preserved, while high-eigenvalue modes corresponding to rapid neighbor-to-neighbor fluctuations are attenuated. Thus iMRIL performs Laplacian smoothing implicitly, reducing variance and enforcing local consistency without tuning λ . $\square$

This implicit smoothing interpretation does not remove the need to learn a policy. Instead, it constrains the class of functions expressible after aggregation. The neighbor-conditioned network f_θ learns how expert actions change under local perturbations and proposes locally adapted actions for each neighbor; the aggregation operator then reduces variance by smoothing these proposals over the retrieved neighborhood. Learning supplies local accuracy, while aggregation supplies stability.

2.4 A PRACTICAL ALGORITHM: DIFFERENCE-AWARE RETRIEVAL POLICIES

We now instantiate the preceding implicit-regularization principle as DARP. For each query state s_q , the algorithm retrieves $\mathcal{N}_k(s_q)$ from the expert dataset and constructs, for each neighbor, a tuple $(s_i^*, a_i^*, \Delta s_i)$ with $\Delta s_i = s_i^* - s_q$. The predictor f_θ maps each tuple to a candidate action a'_i . The relative vector Δs_i is the key representation: it specifies how the retrieved expert state differs from the query state and allows the model to transform the neighbor’s expert action into an action appropriate for the query. In the basic implementation, these candidate actions are averaged.

Retrieval and representation. The neighborhood set $\mathcal{N}_k(s_q)$ can be constructed using any task-appropriate distance metric or representation space. For low-dimensional state inputs, we use Euclidean distance in state space; for image-based inputs, retrieval can be performed in an embedding space produced by a visual representation model. Appendix Section A.2.1 gives additional design details for constructing retrieval neighborhoods. Given the retrieved set, DARP predicts an action for query state s_q via the neighborhood aggregation operator described in Section 2.3:

$$\hat{a}_q = f_{\text{DARP}}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i \quad (3)$$

$$= \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q). \quad (4)$$

At training time, the expert dataset $\mathcal{D}^*$ defines the standard imitation-learning objective:

$$\arg \min_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\|f_{\text{DARP}}(s_q^*) - a_q^*\|^2] \quad (5)$$

The optimization variables are the parameters of f_θ , and the loss minimizes the discrepancy between the aggregated prediction $\hat{a}_q$ and the expert action a_q^* . Because the objective is unchanged from supervised imitation learning, f_θ can be instantiated with standard feedforward or convolutional neural networks. Section 3 shows that the difference-based parameterization is essential for the observed performance gains. At inference time, the policy executes by retrieving k -NN for the current state and evaluating the aggregation rule in Eq. 3.

2.4.1 GOING BEYOND LINEAR AGGREGATION

The averaging rule $\hat{a}_q = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i$ is one member of a broader class of permutation-invariant aggregation functions $g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q)})$. For example, g_ψ may be parameterized by set-compatible neural architectures such as the set transformer (Lee et al., 2019) or DeepSets (Zaheer et al., 2017). This yields the generalized prediction rule $\hat{a}_q = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$. In addition to increasing expressivity, a parametric aggregator g_ψ can represent action distributions richer than the Gaussian distribution implicit in the L_2 regression objective in Eq. 5; examples include Gaussian mixture models (Pignat & Calinon, 2019) and diffusion models (Chi et al., 2024). Instead of predicting a point action $\hat{a}_q$, DARP can predict parameters α of an action distribution $p(a_q; \alpha)$, such as mixture means, covariances, and weights for a Gaussian mixture model, or a score function for a diffusion model. The resulting training criterion is maximum likelihood for multimodal action distributions rather than unimodal L_2 regression:

$$\arg \max_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\log p(a_q^*; \alpha_\theta(s_q^*))], \text{ where } \alpha_\theta(s_q^*) = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q^*)\}_{i \in \mathcal{N}_k(s_q^*)}) \quad (6)$$

For a test-time query state s_q , inference samples $\hat{a}_q \sim p(\cdot; \alpha_\theta(s_q))$ after constructing $\alpha_\theta(s_q) = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$ from the retrieved neighborhood. Appendix Section A.3 provides the corresponding training pseudocode.

2.5 SUMMARY

The preceding analysis shows that enforcing local consistency during behavior-cloning policy fitting is equivalent to manifold regularization through Laplacian smoothing, and that this regularization improves variance, smoothness, and stability relative to standard BC (Theorem 1). Instead of inserting an explicit regularization term with hyperparameter λ into the loss, the same smoothing can be induced architecturally by embedding neighborhood aggregation into the policy (Theorem 2). The resulting algorithm is as follows: during training, for each expert state-action pair (s_q^*, a_q^*) , DARP retrieves the k nearest expert states from $\mathcal{D}^*$, computes difference vectors $\Delta s_i = s_i^* - s_q^*$ from each neighbor to the query, evaluates each tuple $(s_i^*, a_i^*, \Delta s_i)$ with a network f_θ to obtain candidate actions, and aggregates those candidates with a permutation-invariant function g_ψ to produce the final prediction. Training uses only the standard imitation-learning objective.

3 EXPERIMENTAL EVALUATION

We evaluate DARP with respect to three empirical questions: **Q1**: Can DARP consistently outperform standard behavior cloning?, **Q2**: Can DARP handle more complex state representation and action distributions?, and **Q3**: How do different architectural components contribute to DARP’s performance gains? The evaluation spans low-dimensional state inputs, high-dimensional image features, and multiple action-representation regimes. The benchmark suite includes continuous-control tasks from MuJoCo, robotic-manipulation tasks from Robosuite and RoboCasa, and discontinuous environments constructed to stress-test neighborhood-based prediction.

3.1 BASELINE COMPARISONS AND TASK DESCRIPTIONS

MuJoCo Tasks: The MuJoCo (Todorov et al., 2012; Fu et al., 2020) tasks require control of simulated legged embodiments to produce forward locomotion on a flat plane. The evaluated environments are Hopper (single-legged hopping robot), Walker (bipedal humanoid), Ant (quadruped), and HalfCheetah (biped).

Robosuite Tasks: The Robosuite (Zhu et al., 2020) tasks evaluate single-arm object manipulation. Stack requires placing a smaller cube on top of a larger cube; Threading requires controlling a thin, needle-like tool and inserting it through a small ring; and Square Peg requires positioning a square wooden block with a central hole onto a square peg.

RoboCasa Tasks: The RoboCasa (Nasiriany et al., 2024) tasks evaluate single-arm manipulation in randomized kitchen scenes. Drawer requires closing an open drawer, Door requires closing an open microwave oven door, and Stove requires rotating a knob to turn off a stove burner.

Baseline Comparisons: We compare DARP with retrieval, parametric, and regularized baselines: (1) **R&P (Sridhar et al., 2025)**: executes the action associated with the nearest retrieved neighbor; (2) **LWR (Pari et al., 2022)**: performs locally weighted regression over retrieved neighbors; (3) **BC**: denotes standard parametric behavior cloning; (4) **REGENT (Sridhar et al., 2025)**: denotes a transformer-based in-context learner conditioned on retrieved neighbors; and (5) **MRIL**: denotes the explicitly smoothed variant of DARP described in Section 2.2.

3.2 CAN DARP CONSISTENTLY OUTPERFORM STANDARD BEHAVIOR CLONING? (Q1)

This experiment tests the central DARP hypothesis in settings with low-dimensional state representations, where state-space distances are explicit and interpretable. The evaluation includes MuJoCo locomotion and Robosuite/RoboCasa manipulation tasks, with Robosuite data generated using MimicGen (Mandlekar et al., 2023). Unless otherwise specified, the aggregation function g is instantiated as the arithmetic mean of the neighbor action predictions a' .

Method	Hopper	Ant	Walker	HalfCheetah
R&P (Sridhar et al., 2025)	711.82 $\pm$ 85.63	-305.97 $\pm$ 76.42	419.18 $\pm$ 50.21	-178.64 $\pm$ 29.75
LWR (Pari et al., 2022)	1703.78 $\pm$ 245.95	846.59 $\pm$ 216.06	1484.91 $\pm$ 356.54	1945.82 $\pm$ 567.26
BC	2313.65 $\pm$ 203.75	2376.20 $\pm$ 339.43	2658.40 $\pm$ 274.08	1063.23 $\pm$ 371.08
REGENT (Sridhar et al., 2025)	1819.39 $\pm$ 186.24	-302.10 $\pm$ 146.67	507.01 $\pm$ 76.10	169.85 $\pm$ 63.10
MRIL	2793.63 $\pm$ 156.41	3869.08 $\pm$ 241.00	4370.96 $\pm$ 168.13	701.58 $\pm$ 195.08
DARP	3545.57 $\pm$ 3.54	4383.28 $\pm$ 266.37	4894.01 $\pm$ 75.12	5515.41 $\pm$ 841.33
DARP Set Transformer	2965.86 $\pm$ 103.08	4063.79 $\pm$ 218.80	4752.42 $\pm$ 109.23	3417.85 $\pm$ 764.57

Table 1: **Both DARP and DARP Set Transformer outperform the evaluated baselines across MuJoCo domains.** The table reports low-dimensional-state performance for DARP, BC, and retrieval-based baselines. Scores are averages over 100 independent trials with 95% confidence intervals. Parametric policies use Multi-Layer Perceptrons; diffusion-policy results are reported in Section 8.

We find that DARP demonstrates substantial improvements over standard behavior cloning across all tested environments. We observe performance gains ranging from 15-25% points in robotic manipulation tasks and significant score improvements in locomotion tasks (see Table 1 and Table 2). We observe that purely non-parametric methods (R&P and LWR) perform poorly on these tasks, and while MRIL is nearly always able to get a score higher than vanilla BC, the highest scores on this suite of tasks are always achieved by our DARP architecture.

Given the changes introduced for the practical instantiation in Section 2.4, we evaluate whether DARP scales up to higher-dimensional input representations such as images.

3.3 CAN DARP HANDLE MORE COMPLEX STATE REPRESENTATION AND ACTION DISTRIBUTIONS? (Q2)

High-Dimensional Visual Input Representations. To evaluate whether DARP extends beyond compact low-dimensional states, we test it on simulated manipulation tasks using R3M image embeddings (Nair et al., 2022). This setting probes whether retrieval-based prediction remains effective when each state is represented by a

Method	Stack	Thrd.	Peg
BC	44	38	17
DARP	75	76	52

Table 3: Success rates (%) for vision-based Robosuite tasks represented with image features.



Figure 3: The “DrawerClose” RoboCasa task, in which the robotic arm must close an initially open drawer.



Figure 4: The “Threading” Robosuite task, in which the robotic arm must insert a needle-like implement through a small aperture.

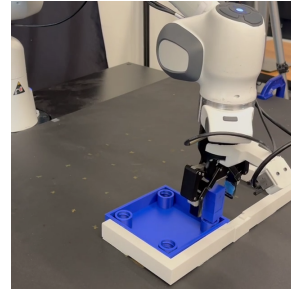


Figure 5: Real-world FurnitureBench square-table assembly task, where the robot must grasp a table leg and screw it into a corner hole on the table-top.

Method	RoboCasa			Robosuite			Real
	Drawer	Door	Stove	Stack	Thrd.	Peg	Sq. Table
BC	54	29	28	47	37	46	44
DARP	85	45	43	72	63	62	92

Table 2: Comparison of DARP and BC across all three evaluation environments using low-dimensional state features. Scores are success percentages (RoboCasa & Robosuite: 100 trials; Real: 50 trials). DARP outperforms BC on every task and environment and more than doubles the BC score in the real-world setting. RoboCasa and Robosuite policies use Multi-Layer Perceptrons, whereas the real-world results use diffusion policy; see Section 8.

high-dimensional visual feature vector (Table 3). DARP outperforms standard BC, and its average improvement ($\sim 35\%$) exceeds the corresponding average improvement on low-dimensional Robosuite states ($\sim 22\%$). These results indicate that the difference-aware retrieval parameterization adapts more effectively than standard BC to complex visual state representations.

Multi-modal Action Distributions. We also evaluate DARP on multimodal imitation-learning problems. In the Push-T environment, DARP exceeds behavior cloning by more than 20%; implementation and evaluation details are provided in Appendix A.2.2.

3.4 HOW DO DIFFERENT ARCHITECTURAL COMPONENTS CONTRIBUTE TO DARP’S PERFORMANCE GAINS? (Q3)

Ablation Study: To isolate the architectural sources of the DARP performance gains, we evaluate the following variants: (1) standard DARP; (2) DARP without neighbor actions; (3) an ensemble of 10 BC agents; (4) DARP with randomly selected neighbors instead of metric-based retrieval; (5) DARP using the L_2 norm of the distance vector; (6) a BC baseline receiving only the query state s_q ; (7) DARP using the query state rather than the query-neighbor difference vector; and (8) DARP with a permutation-dependent, rather than permutation-invariant, aggregator over the a' predictions. Figure 6 reports the resulting ablation scores.

The ablation results identify relative distance vectors and permutation invariance as the dominant architectural factors. Removing neighbor actions produces a smaller degradation, whereas random neighbor selection substantially reduces performance, indicating that metric-consistent retrieval is necessary. Replacing the permutation-invariant aggregation function g with a permutation-dependent alternative also degrades performance, confirming that the set structure of retrieved predictions must be respected.

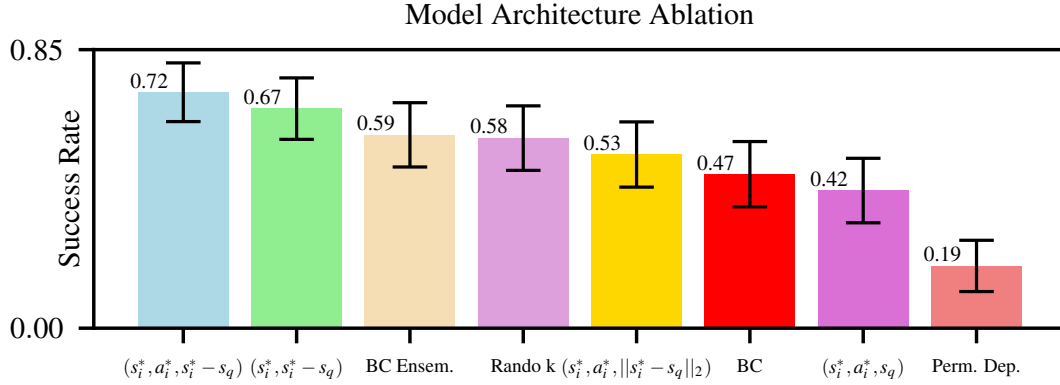


Figure 6: **Distance vectors and permutation-invariant aggregation are primary contributors to DARP performance.** Architectural ablations show how modifying individual components of DARP changes success rate, with the strongest contribution arising from the tuple representation $(s_i^*, a_i^*, s_i^* - s_q)$. Success rates are averaged over 100 Robosuite Stack trials with 95% confidence intervals.

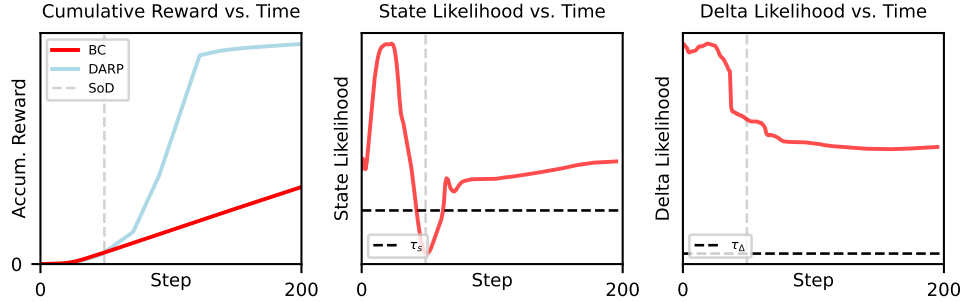


Figure 7: **Cumulative reward traces for BC and DARP on Robosuite Stack show matched early execution followed by divergence.** The vertical dashed line marks the step of divergence (“SoD”), after which BC fails and DARP succeeds. At the SoD, state likelihood is $< \tau_s$ (OOD), whereas delta likelihood is $> \tau_\Delta$ (in distribution).

Divergence Analysis: To characterize why DARP succeeds in rollouts where standard BC fails, we analyze the first point at which the two executions diverge. We define the “step of divergence” as the timestep at which the rewards obtained by DARP and BC become significantly different. We define τ_s and τ_Δ as the 1st percentile of training-set likelihoods; equivalently, 1% of training-time deltas have likelihood below τ_Δ . Across all six analyzed rollouts from two tasks (Robosuite Stack and MuJoCo Hopper), the query state at the SoD has state likelihood $< \tau_s$ but delta likelihood $> \tau_\Delta$. This pattern supports the hypothesis that DARP improves prediction on mildly out-of-distribution states by representing the query through difference vectors to retrieved neighbors. Figure 7 visualizes reward drift, SoDs, and state/delta likelihoods for one task; Appendix A.2.4 provides additional robustness experiments.

4 RELATED WORK

Non-Parametric Imitation Learning Methods: Non-parametric imitation-learning algorithms exploit local dataset structure to generate predictions without compressing all demonstrations into a single global parametric mapping. VINN Pari et al. (2022) studies locally weighted regression for imitation and reports strong performance in image-embedding spaces, while MiDiGaP von Hartz et al. (2025) uses mixtures of Gaussian processes to model multimodal trajectories and enable rapid generalization. SEABO Lyu et al. (2024) applies retrieval in offline RL by assigning rewards to transitions that remain close to retrieved neighbors. FlowRetrieval Lin et al. (2024), STRAP Memmel et al. (2025), and Behavior Retrieval Du et al. (2023) combine non-parametric retrieval with finetuning on large unlabeled datasets, thereby enabling generalization through test-time training. DARP differs from these methods by representing retrieved evidence as $(s_i^*, a_i^*, s_i^* - s_q)$ tuples and learning a semi-parametric policy rather than relying solely on non-parametric aggregation or test-time adaptation. This design combines the variance reduction associated with local methods with the generalization capacity of parametric policies.

Smoothness-Constrained Policy Learning: Recent work has imposed explicit smoothness constraints to improve policy stability and robustness. L2C2 Kobayashi (2022) studies model-free RL under local Lipschitz-continuity constraints, obtaining smoothness and noise robustness without sacrificing expressiveness, and Asadi et al. (2018) develop related Lipschitz-constrained methodology for model-based RL models. CCIL Ke et al. (2024a) extends this line of work to imitation learning by generating synthetic corrective labels with a Lipschitz-constrained dynamics model. Similar principles have also been scaled to humanoid controllers Chen et al. (2024) to reduce deployment-time shakiness. DARP differs by inducing smoothness implicitly through an architectural change while preserving the standard imitation-learning objective.

In-Context Learning Methods: Recent retrieval-augmented work has also framed imitation learning as an in-context prediction problem. REGENT Sridhar et al. (2025) studies retrieval-augmented generalization by conditioning a causal transformer on retrieved states, actions, and rewards, while DPT Lee et al. (2023) uses supervised transformer pretraining to predict actions from query states and in-context datasets, effectively learning an exploration procedure. Other in-context architectures include ICRT Fu et al. (2024), Instant Policy Vosylius & Johns (2025), and KAT Di Palo & Johns (2024). These methods primarily target rapid adaptation to new tasks and environments; DARP instead focuses on improving performance and stability within the standard imitation-learning setting.

5 CONCLUSION

We introduced **Difference-Aware Retrieval Policies for Imitation Learning (DARP)** (DARP), a nearest-neighbor-based imitation-learning algorithm that reparameterizes policy prediction through relative differences between query states and retrieved expert neighbors rather than through a direct global state-to-action mapping. The analysis shows that this architecture implicitly implements Laplacian smoothing. The empirical evaluation across continuous-control and robotic-manipulation domains supports three conclusions. First, DARP consistently improves over standard behavior cloning with low-dimensional state representations. Second, DARP remains effective across different state representations, action-distribution parameterizations, and task complexities, with observed improvements of 15-46% across the tested settings. Third, architectural ablations identify relative distance vectors and permutation-invariant aggregation as essential components of the method.

6 REPRODUCIBILITY STATEMENT

Supplementary source code, policy and environment configuration files, MuJoCo experiment data, scripts for generating Robosuite expert demonstrations with MimicGen, and modality-transformation code will be released upon acceptance. The reported results are reproducible on NVIDIA L40 and L40s GPUs, except for experiments that use transformer components (REGENT and Set Transformer); those components are non-deterministic and may yield small numerical deviations from the reported values.

REFERENCES

- Kavosh Asadi, Dipendra Misra, and Michael Littman. Lipschitz continuity in model-based reinforcement learning. In *International Conference on Machine Learning*. PMLR, PMLR, 2018.
- Mikhail Belkin and Partha Niyogi. Towards a theoretical foundation for laplacian-based manifold methods. *Journal of Computer and System Sciences*, 74(8):1289–1308, 2008.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- Zixuan Chen, Xialin He, Yen-Jen Wang, Qiayuan Liao, Yanjie Ze, Zhongyu Li, S. Shankar Sastry, Jiajun Wu, Koushil Sreenath, Saurabh Gupta, and Xue Bin Peng. Learning smooth humanoid locomotion through lipschitz-constrained policies. *arxiv preprint arXiv:2410.11825*, 2024.

- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. doi: 10.1090/cbms/092.
- Michael Jae-Yoon Chung, Maxwell Forbes, Maya Cakmak, and Rajesh PN Rao. Accelerating imitation learning through crowdsourcing. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4777–4784. IEEE, 2014.
- Norman Di Palo and Edward Johns. Keypoint action tokens enable in-context imitation learning in robotics. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- Maximilian Du, Suraj Nair, Dorsa Sadigh, and Chelsea Finn. Behavior retrieval: Few-shot imitation learning by querying unlabeled datasets. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning, 2020.
- Letian Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, William Chung-Ho Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. In-context imitation learning via next-token prediction. *arXiv preprint arXiv:2408.15980*, 2024.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha Srinivasa, and Abhishek Gupta. Ccil: Continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha S. Srinivasa, and Abhishek Gupta. CCIL: continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024b. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Taisuke Kobayashi. L2c2: Locally lipschitz continuous constraint towards stable and smooth reinforcement learning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4032–4039. IEEE, IEEE, October 2022.
- Jonathan Lee et al. Supervised pretraining can learn in-context reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 43057–43083, 2023.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 3744–3753, 2019.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020. URL <https://arxiv.org/abs/2005.01643>.
- Li-Heng Lin, Yuchen Cui, Amber Xie, Tianyu Hua, and Dorsa Sadigh. Flowretrieval: Flow-guided data retrieval for few-shot imitation learning. In Pulkrit Agrawal, Oliver Kroemer, and Wolfram Burgard (eds.), *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pp. 4084–4099. PMLR, 2024. URL <https://proceedings.mlr.press/v270/lin25a.html>.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR) 2023*, 2023. URL <https://arxiv.org/abs/2210.02747>. Originally appeared as arXiv preprint arXiv:2210.02747.
- Jiafei Lyu, Xiaoteng Ma, Le Wan, Runze Liu, Xiu Li, , and Zongqing Lu. Seabo: A simple search-based method for offline imitation learning. In *International Conference on Learning Representations*, 2024.

- Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Ireaiyo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.
- Elman Mansimov and Kyunghyun Cho. Simple nearest neighbor policy method for continuous control tasks. In *International Conference on Learning Representations (ICLR) 2018*, 2018. Under review as a conference paper at ICLR 2018.
- Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. Strap: Robot sub-trajectory retrieval for augmented policy learning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In Karen Liu, Dana Kulic, and Jeffrey Ichnowski (eds.), *Conference on Robot Learning, CoRL 2022, 14-18 December 2022, Auckland, New Zealand*, volume 205 of *Proceedings of Machine Learning Research*, pp. 892–909. PMLR, 2022. URL <https://proceedings.mlr.press/v205/nair23a.html>.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.
- Jyothish Pari, Nur Muhammad Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation. In *Robotics: Science and Systems (RSS)*. Robotics: Science and Systems Foundation, June 2022. doi: 10.15607/RSS.2022.XVIII-052.
- Emmanuel Pignat and Sylvain Calinon. Bayesian gaussian mixture model for robotic policy imitation. *IEEE Robotics and Automation Letters*, 2019. URL <https://arxiv.org/pdf/1904.10716.pdf>. Preprint version.
- Dean Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Comput.*, 3(1):88–97, 1991. doi: 10.1162/NECO.1991.3.1.88. URL <https://doi.org/10.1162/neco.1991.3.1.88>.
- Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 661–668. JMLR Workshop and Conference Proceedings, 2010.
- Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Geoffrey J. Gordon, David B. Dunson, and Miroslav Dudík (eds.), *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011*, volume 15 of *JMLR Proceedings*, pp. 627–635. JMLR.org, 2011. URL <http://proceedings.mlr.press/v15/ross11a/ross11a.pdf>.
- Steven L. Salzberg and David W. Aha. Learning to catch: Applying nearest neighbor algorithms to dynamic control tasks. In P. Cheeseman and R. W. Oldford (eds.), *Selecting Models from Data: Artificial Intelligence and Statistics IV*, volume 89 of *Lecture Notes in Statistics*, pp. 321–328. Springer, 1994. doi: 10.1007/978-1-4612-2660-4_33. URL https://link.springer.com/chapter/10.1007/978-1-4612-2660-4_33.
- Jonathan C. Spencer, Sanjiban Choudhury, Arun Venkatraman, Brian D. Ziebart, and J. Andrew Bagnell. Feedback in imitation learning: The three regimes of covariate shift. *CoRR*, abs/2102.02872, 2021. URL <https://arxiv.org/abs/2102.02872>.
- Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. REGENT: A retrieval-augmented generalist agent that can act in-context in new environments. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. Oral presentation.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 5026–5033. IEEE, 2012.

-
- Arun Venkatraman, Martial Hebert, and J. Andrew Bagnell. Improving multi-step prediction of learned time series models. In Blai Bonet and Sven Koenig (eds.), *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, pp. 3024–3030. AAAI Press, 2015. doi: 10.1609/AAAI.V29I1.9590. URL <https://doi.org/10.1609/aaai.v29i1.9590>.
- Jan Ole von Hartz, Adrian Röfer, Joschka Boedecker, and Abhinav Valada. The unreasonable effectiveness of discrete-time gaussian process mixtures for robot policy learning. *arXiv preprint arXiv:2505.03296*, 2025.
- Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan R Salakhutdinov, and Alexander J Smola. Deep sets. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 30*, pp. 3391–3401. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/6931-deep-sets.pdf>.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 16, 2003.
- Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Soroush Nasiriany, Yifeng Zhu, and Kevin Lin. robosuite: A modular simulation framework and benchmark for robot learning. In *arXiv preprint arXiv:2009.12293*, 2020.

A APPENDIX

A.1 LEMMAS AND PROOFS

A.1.1 PROOF OF THEOREM 1

The proof of Theorem 1 begins with a standard graph-Laplacian characterization of neighborhood smoothness (Chung, 1997; Zhou et al., 2003; Belkin & Niyogi, 2008).

Lemma 1 (Smoothness regularizer as k -NN graph Laplacian penalty). *Let $\{s_1^*, \dots, s_n^*\}$ be the expert states with corresponding predicted actions $f(s_i^*) \in \mathbb{R}^{d_a}$. For each i , let $\mathcal{N}_k(s_i^*)$ denote the indices of the k -nearest neighbors of s_i^* (excluding i). Define asymmetric weights*

$$\tilde{W}_{ij} = \begin{cases} w_j(s_i^*), & \text{if } j \in \mathcal{N}_k(s_i^*), \\ 0, & \text{otherwise,} \end{cases}$$

and construct a symmetric affinity matrix

$$W_{ij} = \frac{1}{2} (\tilde{W}_{ij} + \tilde{W}_{ji}).$$

Let D be the degree matrix with $D_{ii} = \sum_j W_{ij}$, and define the k -NN graph Laplacian $L = D - W$.

Then the smoothness regularizer can be written as the quadratic form

$$\mathcal{L}_S(f) = \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{N}_k(s_i^*)} w_j(s_i^*) \|f(s_i^*) - f(s_j^*)\|^2 \propto \text{Tr}(F^\top L F),$$

where $F = [f(s_1^*), f(s_2^*), \dots, f(s_n^*)]^\top \in \mathbb{R}^{n \times d_a}$. Equivalently, in the scalar case,

$$\mathcal{L}_S(f) \propto f^\top L f.$$

Corollary 1 (Continuum limit of smoothness regularizer). *Assume states $\{s_i^*\}_{i=1}^n$ are sampled i.i.d. from a smooth density $p(s^*)$ supported on an m -dimensional C^2 manifold $\mathcal{M} \subset \mathbb{R}^d$. Let W be the symmetrized k -NN affinity matrix constructed from a kernel K_Δ with bandwidth h , and let $L = D - W$ be the graph Laplacian.*

If $n \rightarrow \infty$, $h \rightarrow 0$, and $nh^{m+2} \rightarrow \infty$, then the normalized quadratic form converges to the weighted Dirichlet energy:

$$\frac{1}{n^2 h^{m+2}} \text{Tr}(F^\top L F) \longrightarrow C_K \int_{\mathcal{M}} \|\nabla_{\mathcal{M}} f(s)\|_2^2 p(s)^2 d\text{vol}(s),$$

where $C_K > 0$ is a constant depending only on the kernel K_Δ .

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: *The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.*

(ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.

(iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^3$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ with smaller constant.

Proof. We establish the three claims sequentially.

(i) Variance reduction. The Laplacian penalty in $\mathcal{L}_{\text{MRIL}}$ is

$$\sum_{i,j} W_{ij} \|f(s_i^*) - f(s_j^*)\|^2 = 2f^\top Lf,$$

where $L = D - W$ is the graph Laplacian, and W is the k -NN affinity matrix. By Lemma 1, this equals the empirical Dirichlet energy of f on the k -NN graph. It is well known (Zhou et al., 2003; Belkin & Niyogi, 2008) that such a quadratic penalty is equivalent to Tikhonov regularization with respect to the graph Laplacian norm $\|f\|_L^2 = f^\top Lf$. In statistical learning theory, adding a Tikhonov penalty strictly reduces the variance of the estimator compared to the unregularized solution while keeping the bias term controlled. Thus MRIL enjoys smaller estimator variance than vanilla BC, which uses no such penalty.

(ii) Smoothness guarantee. Consider the continuum limit (Corollary 1): for i.i.d. samples $\{s_i^*\}$ from density p on a smooth manifold $\mathcal{M}$, the normalized penalty converges to

$$\int_{\mathcal{M}} \|\nabla f(s)\|^2 p(s)^2 d\text{vol}(s).$$

This is the weighted Dirichlet energy of f on $\mathcal{M}$. If this integral is finite, f belongs to the Sobolev space $H^1(\mathcal{M}, p^2)$, and in particular f is locally Lipschitz almost everywhere with

$$\|f(s^*) - f(s'^*)\| \leq C \|s^* - s'^*\| \quad \text{for } p\text{-a.e. neighbor pairs } s^*, s'^*.$$

Therefore minimizers of $\mathcal{L}_{\text{MRIL}}$ have uniformly bounded local Lipschitz constants along high-density regions of the state space. By contrast, minimizers of vanilla BC have no such constraint: any oscillatory interpolant that matches the training data exactly yields the same supervised risk, so arbitrarily large Lipschitz constants are possible.

(iii) Policy stability. Let $\Delta_t = \|s_t - s_t^*\|$ denote the deviation at time t . For Lipschitz dynamics T ,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|.$$

Decompose the action error:

$$\|f(s_t) - f^*(s_t^*)\| \leq \|f(s_t) - f^*(s_t)\| + \|f^*(s_t) - f^*(s_t^*)\|.$$

For vanilla BC, the first term $\|f(s_t) - f^*(s_t)\|$ is only minimized on the empirical distribution $P_{\mathcal{S}}$; off-distribution, it may be $O(1)$ regardless of Δ_t . The second term satisfies $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Hence the recursion can take the form

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(1) + O(\Delta_t)),$$

which accumulates linearly in t .

For MRIL, the Laplacian penalty enforces

$$\|f(s^*) - f(s'^*)\| \leq C \|s^* - s'^*\| \quad \text{for neighbor pairs } (s^*, s'^*),$$

as shown in part (ii). Thus $\|f(s_t) - f^*(s_t)\| = O(h^2)$ (where h is the radius of the kernel bandwidth around the point s_t) by local-linear regression error bounds, and $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Combining these,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(h^2) + O(\Delta_t)).$$

Since the constant multiplying Δ_t is strictly smaller under the smoothness constraint, the cumulative error grows strictly slower than in vanilla BC. In particular, error growth is sublinear in the rollout horizon when h is small, whereas it is linear for vanilla BC.

³ L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

Conclusion. Claims (i)–(iii) establish that MRIL yields lower variance, uniform smoothness control, and sublinear rollout error accumulation compared to vanilla BC, completing the proof. $\square$

Kernel choice. For the IC smoothness regularizer, we use a Gaussian kernel

$$w_i(s^*) \propto \exp\left(-\frac{\|s^* - s_i^*\|^2}{2h^2}\right), \quad \sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) = 1,$$

with bandwidth h set to the median distance to the k -th nearest neighbor across the dataset. This conventional manifold-regularization choice (Belkin & Niyogi, 2008; Zhou et al., 2003) makes the weights reflect local geometry at the empirical data scale and ensures that the graph Laplacian penalty converges to the Dirichlet energy in the continuum limit. Empirically, this default is stable across tasks, although alternative kernels, including uniform k -NN and exponential decay, produce qualitatively similar behavior.

A.1.2 PROOF OF THEOREM 2

We first state the lemmas needed to express explicit Laplacian regularization and neighbor aggregation in spectral form.

Lemma 2 (Spectral form of explicit Laplacian regularization). *Let L be the symmetric normalized graph Laplacian with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$. The minimizer of the penalized objective*

$$\mathcal{L}_\lambda(f) = \|f - a^*\|^2 + \lambda f^\top L f$$

has the closed-form expansion

$$f_\lambda = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j.$$

Thus λ directly determines the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$ applied to each Laplacian mode.

Proof. Diagonalize $L = U \Lambda U^\top$ with $U = [u_1, \dots, u_n]$ orthogonal and $\Lambda = \text{diag}(\mu_1, \dots, \mu_n)$. Write $f = U c$, $a^* = U b$ in this basis. The objective becomes

$$\|U c - U b\|^2 + \lambda c^\top \Lambda c = \sum_{j=1}^n (c_j - b_j)^2 + \lambda \mu_j c_j^2.$$

Minimizing each term yields $c_j = \frac{1}{1 + \lambda \mu_j} b_j$. Transforming back gives the stated expansion. $\square$

Lemma 3 (Spectral form of neighbor aggregation). *Let $S = D^{-1}A$ be the random-walk matrix of the k -NN graph, with adjacency A and degree D . For any prediction vector f , the neighbor-averaged prediction is $\hat{f} = S f$. In the Laplacian eigenbasis, this corresponds to the spectral filter*

$$\hat{f} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j,$$

i.e. $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

Proof. By definition, $L = I - D^{-1/2} A D^{-1/2}$ and $S = D^{-1} A = I - L_{\text{rw}}$ where $L_{\text{rw}} = D^{-1} L$ is the random-walk Laplacian. Since L_{rw} and L share the same spectrum up to similarity transform, the eigenbasis $\{u_j\}$ diagonalizes S . Thus for each mode u_j , $S u_j = (1 - \mu_j) u_j$, yielding the claimed spectral filter. $\square$

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL’s neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that *iMRIL* is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof. From Lemma 3, the neighbor aggregation operator $S = D^{-1}A$ acts on Laplacian eigenmodes u_j as

$$Su_j = (1 - \mu_j)u_j,$$

where μ_j are the normalized Laplacian eigenvalues. Thus in the graph Fourier basis, neighbor aggregation corresponds to multiplying each mode by the fixed spectral filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

On the other hand, Lemma 2 shows that explicit Laplacian regularization with parameter λ yields the spectral filter $\phi_\lambda(\mu) = (1 + \lambda\mu)^{-1}$. Both filters downweight high-frequency modes ($\mu \gg 0$) while preserving low-frequency modes ($\mu \approx 0$). The key difference is that $\phi_\lambda(\mu)$ requires tuning λ , whereas $\phi_{\text{DARP}}(\mu)$ is parameter-free.

To see the equivalence, note that for small μ ,

$$\phi_{\text{DARP}}(\mu) = 1 - \mu \approx (1 + \mu)^{-1} = \phi_{\lambda=1}(\mu) \quad \text{up to } O(\mu^2) \text{ terms.}$$

Thus DARP can be interpreted as performing Laplacian smoothing with an effective regularization weight of order $\lambda \approx 1$ in normalized units. Moreover, for large μ , $\phi_{\text{DARP}}(\mu)$ damps high-frequency modes even more strongly by driving them toward zero, providing a sharper low-pass effect than explicit regularization.

Therefore, DARP’s aggregation step is mathematically equivalent to implicit Laplacian regularization with fixed spectral filter ϕ_{DARP} , eliminating the need to tune λ explicitly. $\square$

DARP can therefore be viewed as a form of *locally adaptive implicit regularization*: rather than introducing an explicit global weight λ , its neighbor aggregation step enforces smoothness automatically through the graph structure. The effective regularization strength varies with local degree and neighborhood geometry, adapting to the density of the expert demonstrations. Spectrally, this corresponds to the fixed filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$, which suppresses high-frequency modes more aggressively than any fixed explicit λ . Figure 8 illustrates this comparison, showing how DARP achieves sharper low-pass filtering without the need for hyperparameter tuning.

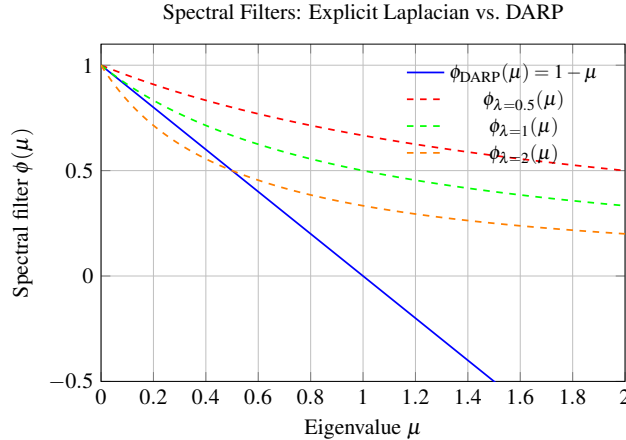


Figure 8: DARP achieves sharper low-pass filtering

A.2 ADDITIONAL EXPERIMENTAL DETAILS

A.2.1 RETRIEVAL

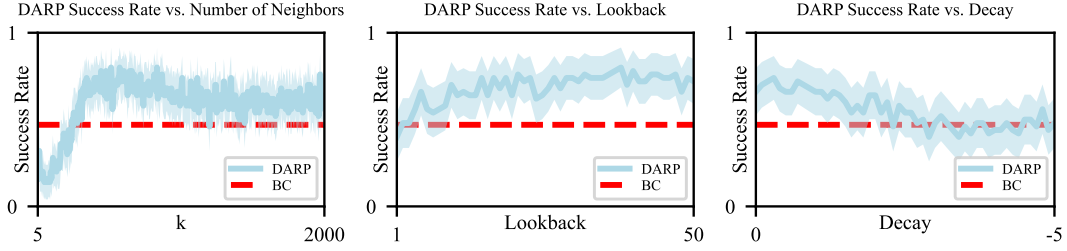


Figure 9: Retrieval-hyperparameter sweep for DARP: (left) performance is low with very few neighbors, reaches a global optimum near 500 retrieved neighbors, and plateaus slightly above BC as k approaches the dataset size; (center) performance generally improves as additional history is included and falls below BC only when little or no history is used; (right) performance is sensitive to the exponential decay applied to older observations. When this decay is too large, DARP behaves similarly to a model with little or no lookback and underperforms BC. Success rate is measured over 50 Robosuite Stack trials with 95% confidence intervals.

The selection of the distance function $d(s_q, s_i^*)$ to select k neighbors is crucial. While we find that simple Euclidean distance between states can work, in our experiments, we use a slightly modified algorithm that takes advantage of the fact that we are working with sequences of states and incorporates history in our distance calculation.

Suppose we have a query trajectory $S_q = (\dots, s_{q,-1}, s_{q,0})$ where $s_{q,0}$ is the current query state s_q . Now suppose we want to calculate $d(s_q, s_i^*)$, where s_i^* is some state from the expert dataset. We first find the trajectory this state is from—call this S_j^* —and the index of s_i^* in this trajectory—call this t . Thus, s_i^* can be rewritten as $s_{j,t}^*$. Given some lookback parameter ℓ which denotes how many past states we want to consider, we get:

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\|$$

This quantity sums Euclidean distances between aligned recent states from the query trajectory and the candidate expert trajectory, assuming the required indices are valid. In practice, more recent states should have greater influence on neighbor selection. Therefore, with exponential decay rate $\gamma \geq 0$, we use

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\| \cdot e^{-\gamma n}$$

Figure 9 reports how environment success rate changes under sweeps of these retrieval parameters.

A.2.2 CAN DARP HANDLE TASKS REQUIRING THE REPRESENTATION OF MULTI-MODAL ACTION DISTRIBUTIONS?

We evaluate the capacity of DARP to represent complex action distributions on the Push-T task of Chi et al. (2024), which requires multimodal action prediction (visualized in Figure 10). In this experiment, DARP uses a Set Transformer head that predicts Gaussian Mixture Model parameters. The GMM-headed variant handles multimodality effectively and improves over BC by 22% on Push-T (Q2), as reported in Table 4. This result shows that DARP can be adapted to action-distribution models beyond unimodal regression.

A.2.3 CAN DARP HANDLE DISCONTINUOUS ENVIRONMENTS WHERE NEARBY STATES MAY REQUIRE OPPOSING ACTIONS?

A central concern for neighbor-based methods is their behavior in discontinuous environments, where nearby Euclidean states may require substantially different actions. We therefore construct a stress test using a modified D4RL Umaze environment; Figure 11 provides a visualization.

Even in this deliberately challenging discontinuous environment, DARP achieves a 57% success rate compared to BC’s 25%. (Q3) (see Table 5) This suggests that the distance vectors and

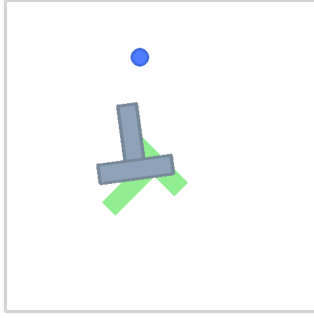


Figure 10: **Push-T Environment.** The goal is to control the blue circle to push the T-shaped block.

Method	Score
BC	48 ± 8
DARP	70 ± 8

Table 4: **Push-T Results.** Averaged over 100 trials, DARP outperforms BC.

permutation-invariant aggregation help the model distinguish between appropriate and inappropriate neighbors, even when spatial proximity doesn't guarantee action similarity.

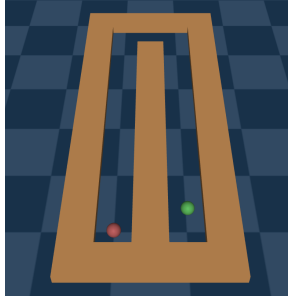


Figure 11: **Long maze environment.** The goal is to move a force-actuated ball from the green start to the red destination.

Method	Succ. (%)
BC	25
DARP	57

Table 5: **Long maze results.** Averaged over 100 trials, DARP significantly outperforms BC.

A.2.4 CAN DARP RECOVER FROM BC ERROR?

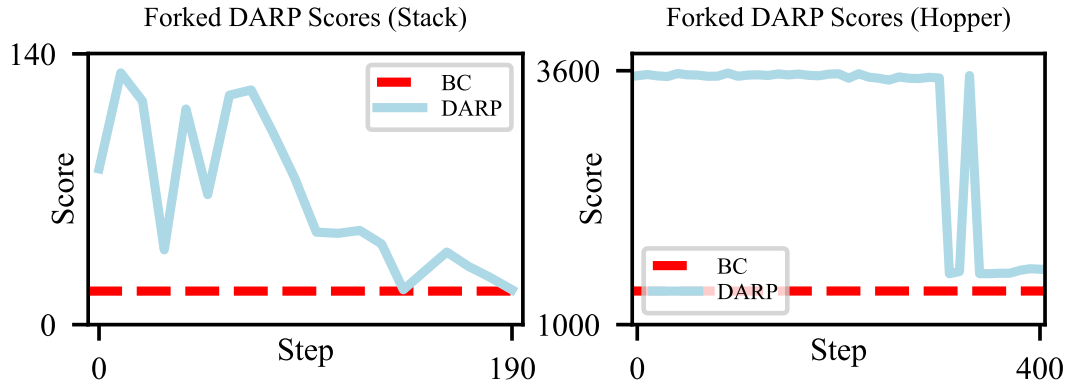


Figure 12: In two different tasks (the Robosuite Stack task and the MuJoCo Hopper task), we rollout a BC agent and create a fork of the environment every k steps (in this case, $k = 10$). Observe that, even as BC nears the end of its failing rollout, DARP is able to scale highly, and is only prevented from doing so about halfway through the Stack rollout and about 80% through the Hopper rollout.

To evaluate robustness to accumulated error, we roll out a BC agent in an environment where it is known to fail and, every k steps, fork the environment state and begin executing a DARP agent in the cloned environment. The results in Fig. 12 show that DARP can recover and obtain high scores even when BC is near failure and has drifted away from the support of expert demonstrations. This supports the claim that DARP is more robust to error accumulation and can act effectively in the mildly out-of-distribution states reached by a failing BC rollout.

A.2.5 COMPARISON WITH CCIL

Environment	Method	Improvement over BC (%)
Hopper	CCIL	32.6%
	DARP	53.2%
Walker	CCIL	84.1%
	DARP	84.1%
Ant	CCIL	12.6%
	DARP	84.5%
HalfCheetah	CCIL	5.4%
	DARP	418.7%

Table 6: **Relative improvement over BC compared to CCIL.** Comparing DARP against CCIL on standard MuJoCo benchmarks. DARP achieves higher or equal relative gains across all tasks.

Table 6 compares DARP with CCIL, a baseline that explicitly induces smoothness. We use the scores reported in the CCIL paper and compare each method by percent improvement over BC. DARP substantially exceeds CCIL on three of four environments, with the largest margin on HalfCheetah (418.7% vs 5.4% improvement).

A.2.6 DISTANCE METRIC SENSITIVITY

Method	Success Rate (%)
BC	47
DARP (Cosine Similarity)	70
DARP (Euclidean Distance)	75

Table 7: **Distance Metric Sensitivity.** Comparison of success rates using R3M features. DARP with Euclidean distance and cosine similarity perform similarly, beating BC by 28% and 23% respectively. Results are on the Robosuite Stack task.

Although the main experiments use Euclidean distance for neighbor retrieval, alternative metrics such as cosine similarity are natural in high-dimensional embedding spaces such as R3M. Table 7 shows that DARP performs similarly with cosine similarity, with only a 5% decrease relative to Euclidean retrieval. This indicates that DARP is not highly sensitive to the specific distance metric in this setting.

A.2.7 DARP IN COMBINATION WITH DIFFUSION POLICY

Although the principal experiments use an MLP backbone, diffusion policy (Chi et al., 2024) is a strong model class for imitation learning, especially in manipulation domains. Table 8 shows that DARP is compatible with diffusion-policy backbones and can produce a stronger combined model. Relative to DARP with an MLP backbone, the diffusion-backed DARP variant increases success rate by 20%, 34%, and 26% on the three tasks, respectively, and exceeds standard BC by 48%, 60%, and 38%.

We also find empirically that DARP with an MLP backbone is substantially faster than standard diffusion, especially at inference time; Table 9 reports this comparison.

Method	Stack	Threading	Peg Insertion
BC (MLP)	34	12	28
DARP (MLP)	62	38	40
Diffusion	52	44	54
DARP w/ Diffusion	82	72	66

Table 8: **DARP in combination with diffusion policy.** DARP provides significant improvements when applied to both standard MLP policies and Diffusion policies. Results are on the Robosuite Stack, Threading, and Peg Insertion tasks as success rate across 50 trials. States are state-based. Note that the number of demonstrations used is much less than that used in Table 2, so baseline BC and DARP numbers do not match.

Method	Training (s/epoch)	Inference (s/step)
Diffusion	5.610	0.15700
DARP (MLP)	0.559	0.00437

Table 9: **Computational efficiency of DARP with an MLP backbone and diffusion policy.** Comparison of runtime costs between DARP (MLP) and diffusion policy. Diffusion policy is significantly more expensive, being $\approx 10\times$ slower in training and $\approx 36\times$ slower during inference.

A.2.8 DARP TRAINED ON HUMAN DEMONSTRATIONS

Method	Success Rate (%)
BC	45
DARP	60

Table 10: **Results with human demonstrations.** Comparison of success rates when training on human data rather than data collected by RL policies. Results are on the Robosuite Stack task.

The MuJoCo and Robosuite demonstrations used in the main experiments are generated by an optimal reinforcement-learning policy. To verify that the performance gain is not specific to policy-generated demonstrations, we also evaluate human demonstration data. On the Robosuite Stack task with human demonstrations, DARP outperforms standard BC by 15% (Table 10).

A.2.9 CHOICE OF RETRIEVAL HYPERPARAMETERS

Environment	Method	Score (Mean)
Hopper	BC	507.49
	DARP	805.69
Stack	BC	0.12
	DARP	0.31

Table 11: **Performance comparison when validation loss is used to select training epochs and retrieval hyperparameters.** Mean scores on Hopper and Stack environments. Observe that DARP maintains a performance gain in comparison to BC.

Figure 9 shows that some retrieval-parameter choices, such as using very few neighbors, can make DARP underperform standard BC. However, selecting retrieval hyperparameters by minimizing validation loss reliably identifies performant settings, as shown in Table 11.

A.3 PSEUDOCODE

Algorithm 1 gives pseudocode for the DARP training procedure.

Algorithm 1 Difference-Aware Retrieval Policies

```

1: Input: Expert demonstrations  $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$ , number of neighbors  $k$ 
2: Initialize:  $f$  parameters  $\theta$ 
3: if  $g$  is parametric then
4:   Initialize:  $g$  parameters  $\psi$ 
5: end if
6: // Training Loop
7: while not converged do
8:   Sample batch of query data  $(s_q^*, a_q^*) \sim \mathcal{D}^*$ 
9:   for each query pair  $(s_q^*, a_q^*)$  in batch do
10:    // Find  $k$ -Nearest Neighbors from the entire dataset  $\mathcal{D}^*$ 
11:     $\mathcal{N}_k(s_q^*) \leftarrow \text{argmin-}k; d(s_q^*, s_j^*)$ 
12:    // Compute Neighbor-based Predictions
13:    for each neighbor index  $i \in \mathcal{N}_k(s_q^*)$  do
14:       $a'_i \leftarrow f_\theta(s_i^*, a_i^*, s_i^* - s_q^*)$ 
15:    end for
16:    // Aggregate Predictions
17:    if  $g$  is parametric then
18:       $\hat{a}_q \leftarrow g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
19:    else
20:       $\hat{a}_q \leftarrow g(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
21:    end if
22:  end for
23:  // Update Parameters based on the batch loss
24:   $\mathcal{L} \leftarrow \sum_{(s_q^*, a_q^*) \in \text{batch}} \|\hat{a}_q - a_q^*\|^2$ 
25:  // Gradient descent step
26:   $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$ 
27:  if  $g$  is parametric then
28:     $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}$ 
29:  end if
30: end while
31: Output: Trained parameters  $\theta$  and, if applicable,  $\psi$ 

```

A.4 RUNTIME ANALYSIS

Environment	k	Train (s/epoch)	Test (s/step)
Hopper (1,000 datapoints)	BC	0.102	0.00127
	100	0.126	0.00413
	200	0.128	0.00418
	300	0.130	0.00420
	400	0.131	0.00421
	500	0.130	0.00419
Stack (4,200 datapoints)	BC	0.431	0.00139
	100	0.556	0.00437
	250	0.539	0.00442
	500	0.559	0.00437
	750	0.576	0.00433
	1000	0.591	0.00478
	1500	0.682	0.00452
	2000	0.758	0.00451

Table 12: **Runtime comparison across environments.** Training and testing speeds (in seconds) for Behavior Cloning (BC) and varying values of k on Hopper and Stack datasets.

Table 12 reports the computational overhead of DARP during both training and inference. The measured cost scales sublinearly with k , and inference remains compatible with real-time robotic control. For example, on Robosuite Stack with $k = 500$, inference requires approximately 0.00437 seconds per step on our hardware, corresponding to a control frequency above 230 Hz.